

Implementation of Arbitrary LSB Encoding for Audio Steganography

Declan McCloskey

Introduction

Steganography is the hiding of information within cover information, essentially using pre-existing data as a medium for communication. Digital audio, as with most digital media, provides many opportunities for covert information transmission in the temporal and frequency domains [1]. One of the challenges to hiding information in an audio signal is of course distortion, as it is expected that a transmitted audio signal be listened to. Excessive distortion due to information hiding would not only mean the original data can't be listened to as intended but could also lead to the investigation of the signal and detection by an unintended recipient. It is therefore necessary for steganography to exploit unnecessary or extraneous data content in the transmission medium of choice to avoid detection. For audio files, this means exploiting the human auditory system by taking advantage of its insensitivity to low magnitude changes or its response to specific frequencies [2].

One of the earliest concepts for audio steganography is least significant bit (LSB) encoding, which takes advantage of the human auditory system's insensitivity to low magnitude noise by overwriting the LSB of time-series data with bits from another message. The goal of this paper is to detail an implementation of LSB encoding that serializes messages into bitstreams, encodes these messages into the N least significant bits of an audio signal, and successfully recreate the original hidden message given knowledge about its characteristics. The focus here will be on implementing a method flexible enough to handle arbitrary bit depth per sample by leveraging a variable number of LSBs. This is an important lever to be able to pull regarding LSB encoding, as your data rate is entirely dependent on the number of points in the modified media. Utilizing more bits in these data points directly correlates to gains in data rate but comes at the cost of distortion. Sending information dense messages such as images is a focus of this project and doing so without distortion would be necessary in a real-world scenario.

Approach

The goal of this implementation is to encode arbitrary bit streams into the LSBs of an audio signal. Although this approach could be broadened to support many audio formats, a 16-bit integer format was chosen as the primary target for manipulation. With the actual sample format defined, we can remain entirely agnostic to the sample rate. This allows for varying data rates, which are nominally equal to the sample rate in the assumed case where each sample

transmits one bit of information [2]. The principal audio example that will be analyzed for demonstration purposes is sampled at 22.05 kHz and is sourced from the University of Illinois Chicago but composed by John Williams [3].

A basic overview of the implementation architecture used in this project can be seen in Figure 1. While a cryptographic technique can be included to further disrupt attempts by unintended recipients to access the encoded message, this part of the approach has been eschewed in this implementation. Verification of the message’s integrity is the most important facet of the initial system’s design and will be the focus.

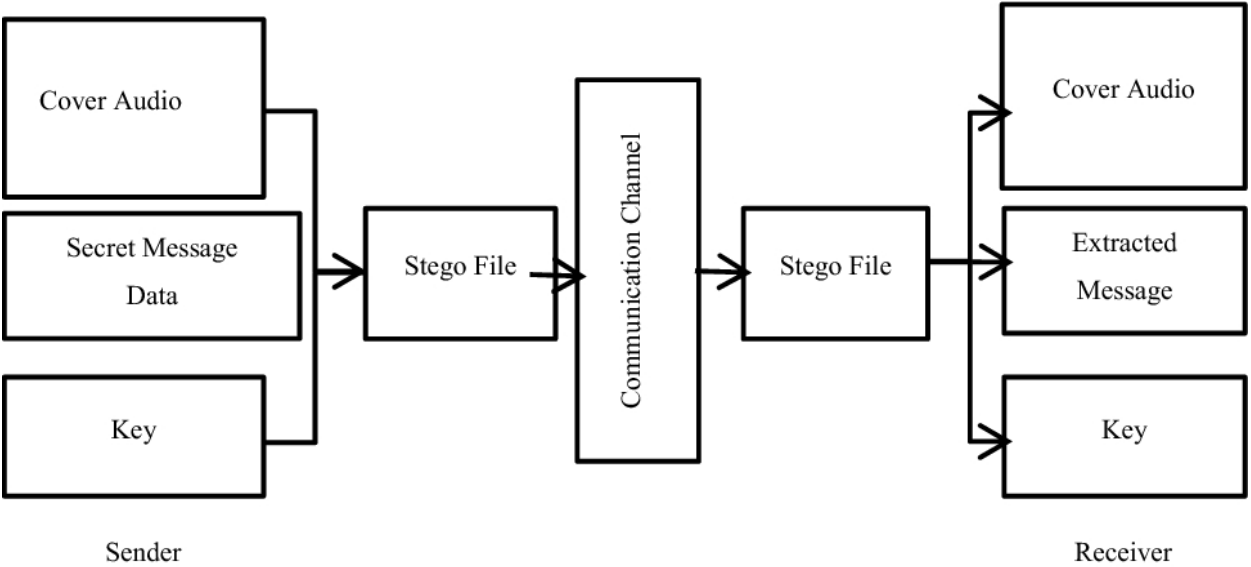


Figure 1: Steganographic Flow [4]

The first step in LSB encoding is serializing the message data and converting it to a stream of bits to be transmitted. Using ASCII/UTF-8 encoding, a text excerpt can be cast to its decimal form by casting to a uint8 and then calling *dec2bin*. With some string padding to ensure all characters take up 8 bits, an arbitrary string of characters can be converted to a stream of bits like those in Table 1. The highlighted text is the portion that the bitstream in the second row represents.

Table 1: Bit stream conversion

Char array	I am a hidden message. You cannot see me, or can you? Repeat: {newline}
Bit Stream	01001001001000000110000101101101001000000110000100100000...

With the message bit stream serialized, the process becomes the same regardless of the initial data type. The 16-bit samples are logically AND’d with -2 cast to an int16, which yields 15 ones followed by a zero in the LSB. The result of this AND will leave the original sample with a zeroized LSB. We can then OR this sample with the int16 representation of the first bit of the bit

stream to obtain the edited form of the file. This process is then repeated until the full length of the message is encoded into the original file. This process is diagrammed in Figure 2.

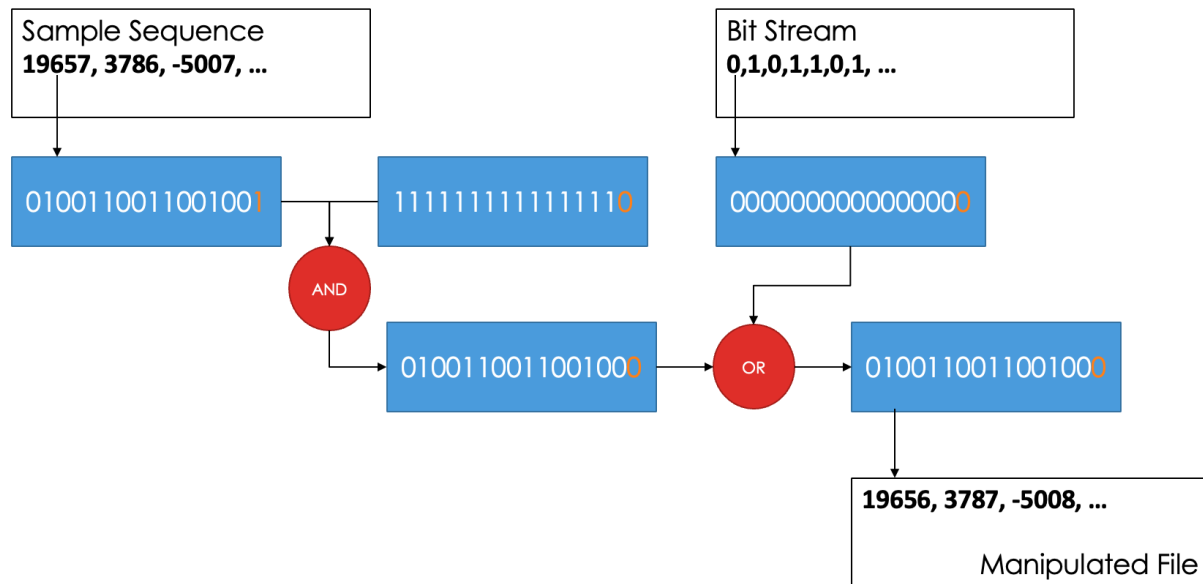


Figure 2: Sample Manipulation Schematic

This process is straightforward but can only encode data at a rate equal to the sample rate. To achieve a higher data rate, more of the original data must be used for transmitting the hidden message. This amounts to using additional bits, expanding our “data field” from the least significant bit to the N least significant bits. This can be done by first modifying our zeroizing AND value with a number of trailing zeros equal to the number of LSBs desired for message transmission. An example of this expanded process with 3 LSBs is displayed in Figure 3.

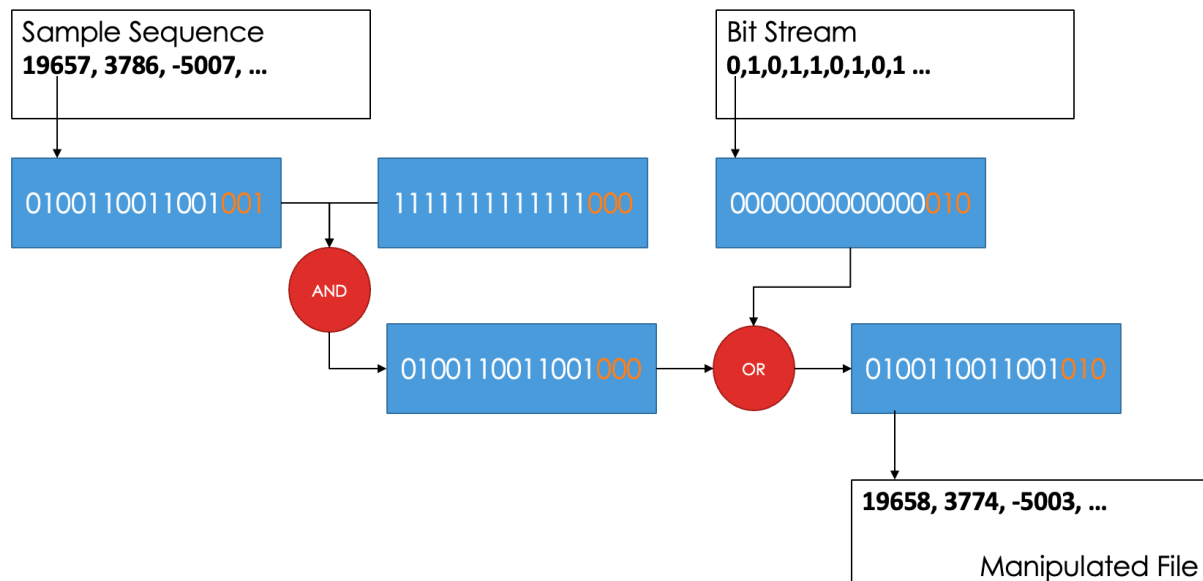


Figure 3 LSB Encoding Process for 3 LSBs

An increased data rate allows for the encoding of more complex messages in the same cover media. The challenge then becomes serializing the data into a format that can be reconstructed, and one such example using images is detailed in the results section of this report.

Decoding the interned data follows a similar process. A bitwise AND of the “received” samples first zeroizes all but the N LSBs we are attempting to extract data from. These int16s are then converted to an N-column char array of bits before being serialized back into a one-dimensional bitstream. This bitstream can then be chunked into bytes and reformatted into the form of the original data. Figure 4 shows this recovery process.

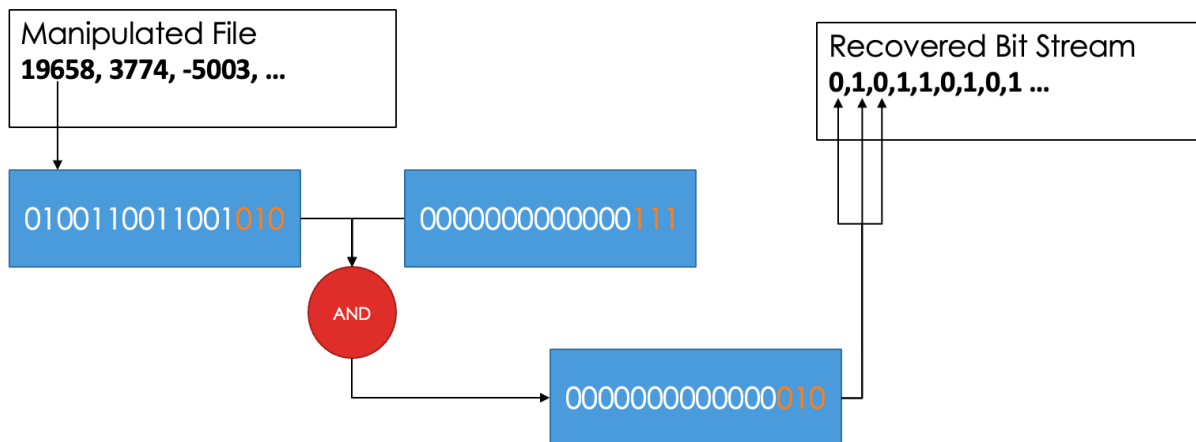


Figure 4: LSB Decoding Process

A major advantage of LSB encoding over other audio steganographic methods is its ease of recovery once a message is encoded. Regardless of what kind of message is being sent or any encryption used, the raw bits being present in a defined data field makes recovery of the message. This limits the need for something like a forward error correction (FEC) code when analyzing soft symbols like in an approach such as tone insertion [2]. This can also be seen as a drawback, as an easily recoverable hidden message means a higher chance of interception.

Another advantage is low distortion, since a 16-bit integer is fairly resistant to interference at the lowest level. When there are 2^{16} (65,536) different levels each sample could be, being off by one or two per sample provides little qualitative difference. A qualitative and quantitative approach to measuring this distortion will be discussed in the results section.

The largest drawback is that LSB encoding provides little to no robustness against lossy filtering or compression techniques. Any resampling completely distorts the hidden message, and most filtering techniques that would change the value of samples has a very high chance of leaving the LSBs altered and the message in an irrecoverable state.

Results

A 60 second 16-bit 22kHz audio recording of the Cantina Band from Star Wars was used as the primary cover audio for this implementation. The LSBs of this audio will be used to encode bitstreams. An excerpt from the song is plotted using Matlab in Figure 5.

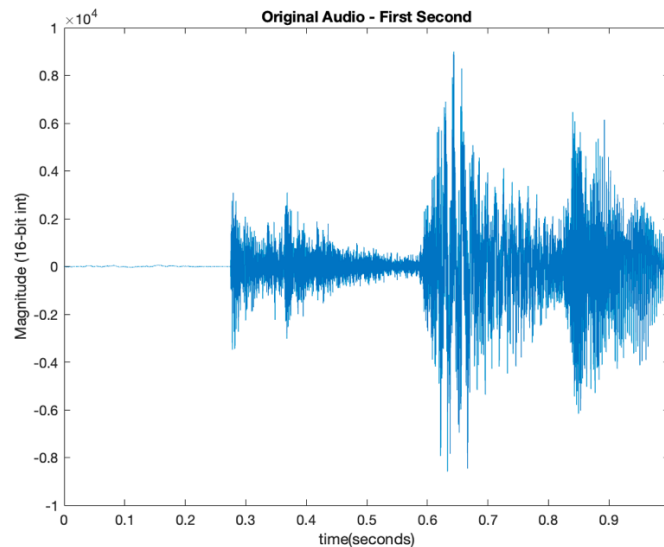


Figure 5: Original Audio

The simplest implementation of this method involves serializing a string, or character array, and encoding it into the LSB of the audio. We can then perform the bitwise logical operations discussed in the approach section and yield an altered audio signal. The encoding process for a single LSB is shown in the function excerpt below, performing the same steps as those in Figure 3.

```
function [output] = encode_audiostream_1D(audio, bitstream)
    %encode a bitstream into the LSB of an audio file
    for i = 1:length(bitstream)
        %set LSB to 0 so we can OR it
        %00...001 is -2 for int16
        audio(i) = bitand(audio(i), int16(-2));
        %set LSB using OR and the bitStream
        audio(i) = bitor(audio(i), typecast(uint16(bin2dec(bitstream(i))), 'int16'));
    end
    output = audio;
end
```

This output can then be written to a WAV file, allowing the message to be sent through any communications channel that supports file transfer. Once at the receiving end, the audio can be listened to without discernable distortion and the secret message can be recovered by using the following function.

```
function [LsbByteArray] = extractLSB(audio,bits)
    %take the LSB from audio and chunk it into bytes the size of
    %the bits argument.
```


LSBs used from two to thirteen. One example of the full process of message storage, retrieval and SNR measurement is shown in Figure 7.

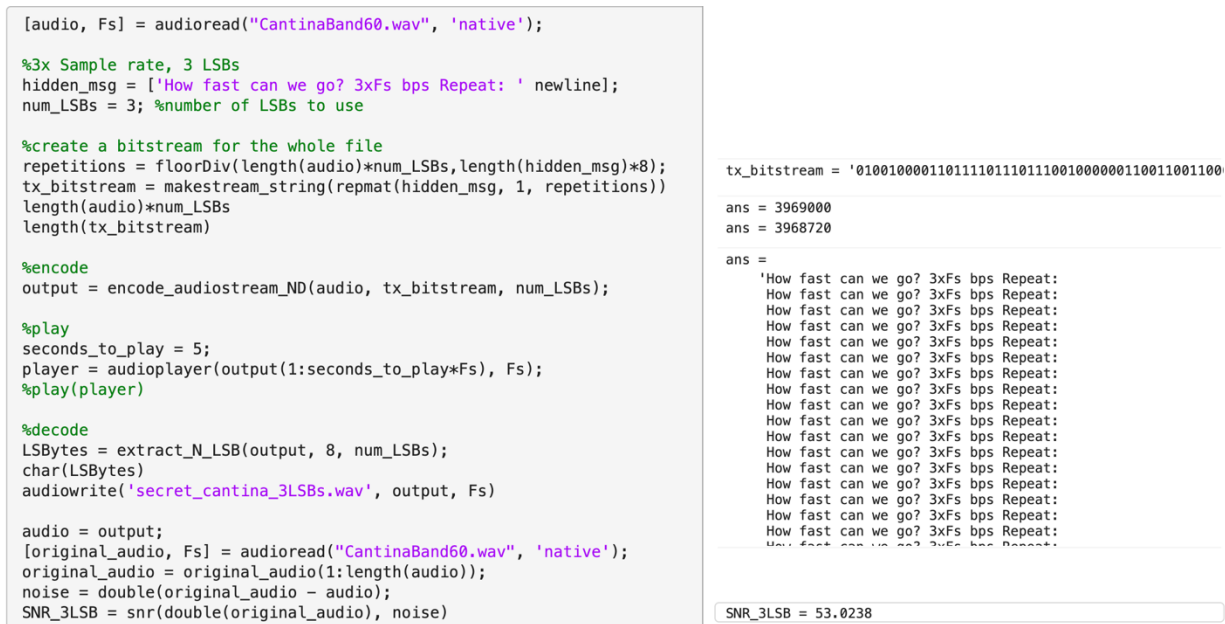


Figure 7: Data Transmission using 3 LSBs

This process was completed for of the aforementioned LSB values. Results of this process are recorded in Table 2. Bit rate was calculated by dividing the number of bits recovered from the file and dividing that by seconds, or the length of the file divided by the sample rate.

Table 2: Table of Quality Results and Data Capacity

Number of LSBs	Message size (bytes)	Bit Rate (kbps)	SNR
1	165375	22050	66.3876
2	330714	44100	59.3887
3	496089	66150	53.0238
4	661466	88200	47.197
5	826841	110250	40.8933
6	992217	132300	35.1120
7	1157593	154350	28.8095
8	1322970	176400	24.0021
9	1488345	198450	16.7563
10	1653716	220500	10.9098
11	1819115	242550	4.5564
12	1984476	264600	-1.8585
13	2149875	286650	-8.7454

As expected, the bit rate of messages increased linearly with the number of LSBs used, and matched multiples of the sample rate. Qualitatively, the audio remains mostly undistorted

to the human ear until the use of around eight LSBs, at which point a faint buzzing can be heard in the audio. The buzzing increases as more LSBs are used before becoming uncomfortable to listen to when using all but three bits for information storage. Seven bits is the optimistic upper limit for maximum data transfer while minimizing interference, as the ~30 dB SNR is a good baseline for listenability both qualitatively and quantitatively. Figure 8 through Figure 10 provide a visualization of the data shown in Table 2.

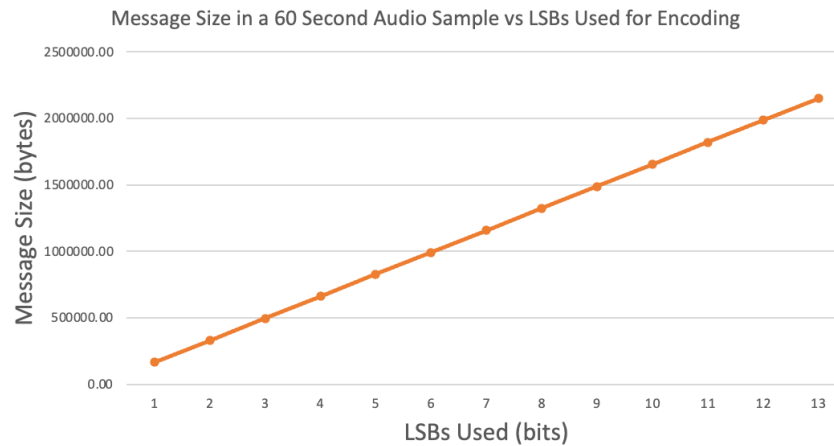


Figure 8

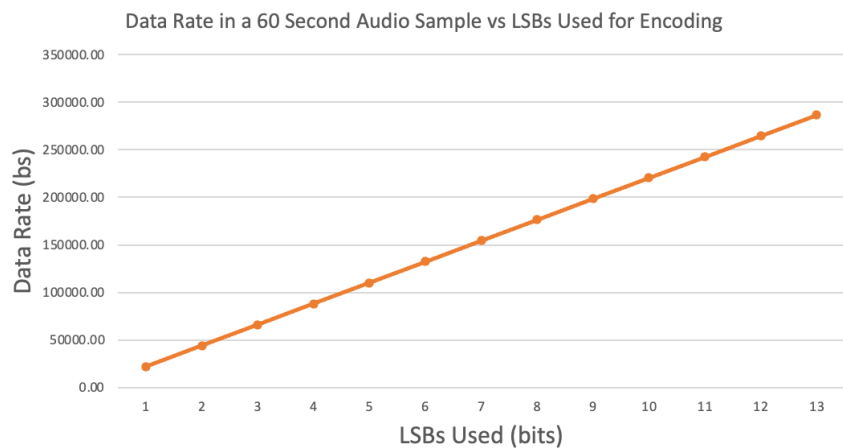


Figure 9

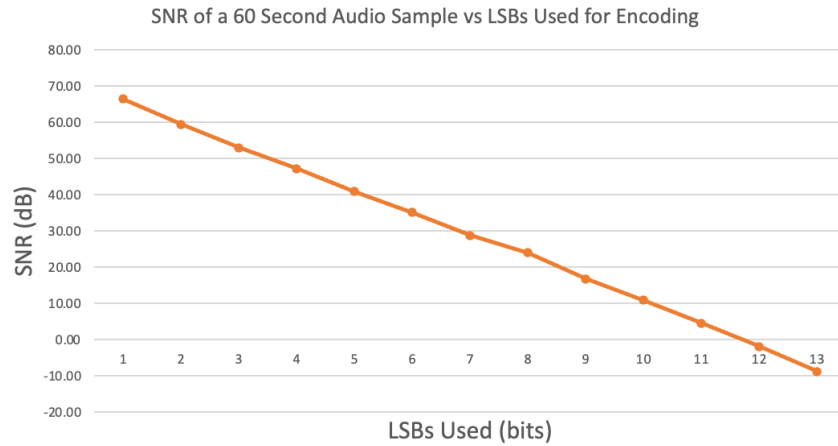


Figure 10

Image Transmission

With a high capacity for information storage, more complex data can be stored in an audio file than just strings of text. With the proper serialization techniques, we can transport any kind of digital data using the LSB encoding method. Figure 11 shows a pile of peppers in a greyscale TIFF image, a classic image used in many DSP publications.



Figure 11: Peppers.tif[5]

With a 512x512 resolution, it takes ~256 kilobytes to represent this image using a single byte for each pixel's greyscale value. The exact number of bits in the image is calculated in the first row of the below equation block. The second row calculates the number of samples in the original audio sample. Because the number of bits exceeds the number of samples, we need to use more than the least significant bit to encode the image. Two LSBs provide enough room in the audio, so that will be sufficient.

$$\begin{aligned}
 512 \times 512 \times 8 \text{ bits} &= 2,150,400 \text{ bits/image} \\
 60\text{s} \times 22050 \text{ Hz} &= 1,323,000 \text{ samples}
 \end{aligned}$$

Serializing the data into a bitstream is the first step to concealing the image in the audio's LSBs. This is accomplished by converting the uint8s of the TIFF to binary strings and

feeding it into a stream of bits column by column. Once it takes the form of a bit stream, we can use the same method as before to encode the image into the audio. Using two LSBs, we encode the serialized data into the audio. It can be extracted as before and then reformatted into a 512x512 matrix of bytes since we know the type of data we expect in the message. The code snippet implementing this process and its results can be seen in Figure 12.

```
%first we need to serialize the image
pep = imread('peppers.tif')
binary_pep = dec2bin(pep);
binary_pep = reshape(binary_pep.', 1, []); %1s and 0s

%now embed into audio using our past function. We can use 2 for LSB
num_LSBs = 2;
peppered_song = encode_audiostream_ND(audio, binary_pep, num_LSBs);
audiowrite('peppered_cantina.wav', peppered_song, Fs)

%play a sample
seconds_to_play = 5;
player = audioplayer(peppered_song(1:seconds_to_play*Fs), Fs);
%play(player)

%extract serialized image bytes
serialized_pep = extract_N_LSB(peppered_song, 8, num_LSBs)
%we know how many bytes we are looking for
bytes_in_image = 512*512;
serialized_pep = serialized_pep(1:bytes_in_image)
reconstructed_pepper = reshape(uint8(serialized_pep), [512, 512])
imshow(reconstructed_pepper)
```

```
serialized_pep = 1x262144
    5    20    20    14    17    13 ...

serialized_pep = 1x262144
    5    20    20    14    17    13 ...

reconstructed_pepper = 512x512 uint8 matrix
    5    65    67    57    66    60 ...
   20    84    89    79    88    84 ...
   20    88    97    86    93    88 ...
   14    86    96    82    86    80 ...
   17    90   101    86    90    87 ...
   13    87    97    82    90    91 ...
    9    81    87    72    83    88 ...
   14    84    88    71    82    87 ...
   10    78    83    84    78    90 ...
   11    79    84    85    78    90 ...
   ...
   ...
```



```
function [hidden_bits] = makestream_string(message)
%create a bitsream given a string of arbitrary length
message = uint8(message);
message = dec2bin(message);
```

Figure 12: Image Serialization, encoding, and decoding

Table 3 shows audio quality results for audio containing the pepper image.

Table 3: Quality metrics for Pepper Encoding

Number of LSBs	Message size (bytes)	Bit Rate (kbps)	SNR
2	262144	44100	58.8070

To show the lack of resilience to filtering techniques, we can apply a simple FIR filter to the audio file containing the pepper image and see what we end up reconstructing. Using much of the same code as in Assignment 8, we construct an FIR filter with a passband edge of 700 Hz and a stopband edge of 1000 Hz. We can then convolve the audio with the filter, cast it back to an int16, and see what happens to the reconstructed image. Figure 13 shows the filtering process and the resultant noise showing that our encoded signal has essentially been wiped out. This is one limitation of LSB encoding.

```

%filtered audio
%this comes from assignment 8, and I've reused much of my code with a
%different sample rate
passband_edge = 700; %Hz
stopband_edge = 1000; %Hz
Fc = 0.5 * (passband_edge + stopband_edge); % Hz
wc_sansPi = 2*(Fc/Fs);
wc = wc_sansPi * pi;
transi_BW = 2*pi* ((stopband_edge-passband_edge)/Fs);
M_min = (12*pi)/transi_BW; %441, good odd length filter

%window it
wc_win = wc;
M = -220:220;
window = blackman(441);
h = sin(wc_win*M)./(pi*M);
h(221) = wc_win/pi;
h_windowed = h .* window';

%filter
filtered_pep = conv(audio,h_windowed, 'same');
serialized_filtered_pep = extract_N_LSB(int16(filtered_pep), 8, 2);
reconstructed_filtered_pep = reshape(uint8(serialized_filtered_pep), [512, 512]);
imshow(reconstructed_filtered_pep)

```



Figure 13: Filtered and Reconstructed Pepper Image

Future Work

While this implementation is effective at encoding data into audio files, there are still many avenues for improvement. Namely, no encryption or FEC was implemented in this approach, meaning that our original message would be stored in plaintext, visible to all who know the encoding scheme. Taking the serialized bitstream and applying any encryption technique would be enough to add another layer of security, and at little cost in terms of data rate. A FEC would compromise data rate but provide some resilience against bit flips or other errors.

Having some leading fields that detail metadata would also be a good feature to implement. For example, some indication of message size and flag to indicate what format it originally took would mean that foreknowledge of the message format was unnecessary. It would add additional computational complexity but is likely a necessity if this were to be implemented for sending complex information like compressed data formats. This would be what I tried to implement first, as getting a good metadata format down combined with arbitrary LSB encoding would allow for a very powerful communication tool.

Trying the technique out with other audio formats would also yield interesting results. In testing, I used 16-bit 44kHz audio and found no difference other than a higher effective data rate. Trying another format like 32-bit floating point audio would provide even more information capacity, but I would need to run similar tests to what was covered in the results section to see if it behaved differently from the integer audio.

Summary

This project served as an excellent learning opportunity for multiple topics. We've worked with audio on several assignments, but this was the deepest dive I've done on an audio-specific topic in this course. It was interesting to research the difference between audio formats and how they are handled by Matlab. Typically, files are normalized to values between $[-1, 1]$, so to get direct access to the actual 16-bit values of the WAV file, it needs to be read in native mode. All the connections to steganographic concepts also made for positive learning

opportunities. The treatment of LSB encoding as noise was something that made the concept of SNR intuitively click while evaluating the produced audio files, as I realized signal distortions could be measured empirically in a case where the distortions are intentional and directly measurable. This was also my first time using the low-level bitwise operators in Matlab, something that is usually painful in a higher level dynamically language, but it was surprisingly straightforward after realizing that binary formats were treated as strings.

The implementation in this paper focused on being able to encode arbitrary bitstreams at a variable bit-depth and successfully decode the bitstreams with limited knowledge of the data format. This was successfully implemented for multiple data formats, including images, enforcing signal processing concepts like sample rates, data rates, distortion, and lossy processing.

References

- [1] Mishra, Shilpi & Yadav, Virendra & Trivedi, (Dr.) Munesh & Shrimali, Tarun. (2018). Audio Steganography Techniques: A Survey. 10.1007/978-981-10-3773-3_56.
- [2] Djebbar, F., Ayad, B., Meraim, K.A. et al. Comparative study of digital audio steganography techniques. J AUDIO SPEECH MUSIC PROC. 2012, 25 (2012). [10.1186/1687-4722-2012-25](https://doi.org/10.1186/1687-4722-2012-25)
- [3] Troy, Pat. "UIC CS101." *CS 101 - Fall 2011*, 2011, www2.cs.uic.edu/~i101/.
- [4] Abdulrazzaq, S.T., Siddeq, M.M. & Rodrigues, M.A. A Novel Steganography Approach for Audio Files. SN COMPUT. SCI. 1, 97 (2020). <https://doi.org/10.1007/s42979-020-0080-2>
- [5] Hao, Pengwei. "QMUL ECS605 - Image Processing." *Index of ~Phao/IP/Labs/CWK/Images*, Queen Mary University of London, 2018, www.eecs.qmul.ac.uk/~phao/IP/Labs/cwk/images/.

Raw Links for Ease of Reference

- [1]https://www.researchgate.net/publication/320124080_Audio_Steganography_Techniques_A_Survey
- [2] <https://asmp-erasipjournals.springeropen.com/articles/10.1186/1687-4722-2012-25>
- [3] <https://www2.cs.uic.edu/~i101/SoundFiles/>
- [4] <https://link.springer.com/article/10.1007/s42979-020-0080-2#Sec4>
- [5] <https://www.eecs.qmul.ac.uk/~phao/IP/Labs/cwk/images/>

Appendix

A: Key Functions

```
function [output] = encode_audiostream_ND(audio, bitstream, n_bits)
    zero_out = repmat('0', 1, n_bits);
    one_out = repmat('1', 1, 16-n_bits);
    binstr = [one_out zero_out];
    ander = typecast(uint16(bin2dec(binstr)), 'int16');
    audio=audio(1:floorDiv(length(bitstream),n_bits));

    for i = 1:length(audio)
        bit_idx_start = (i-1) * n_bits + 1; %3
        bit_idx_end = i*n_bits; %4
        audio(i) = bitand(audio(i), int16(ander));
        %set LSB using OR and the bitStream
        bitsection = bitstream(bit_idx_start:bit_idx_end);
        audio(i) = bitor(audio(i), typecast(uint16(bin2dec(bitsection)), 'int16'));
    end
    output = audio;
end
```

```
function [LsbByteArray] = extract_N_LSB(audio,bits,num_LSBs)
    %take the LSB from audio and chunk it into bytes the size of
    %the bits argument. Do this for N LSBs
    zero_out = repmat('0', 1, 16-num_LSBs);
    one_out = repmat('1', 1, num_LSBs);
    binstr = [zero_out one_out];

    LSBarray = dec2bin(bitand(audio(1:end), typecast(uint16(bin2dec(binstr)),
'int16')));
    %the special case of 8 does not return a full 8 bit binary string, it
    %drops the leading 0. We can send it directly to output
    if num_LSBs == 8
        LsbByteArray = bin2dec(LSBarray)';
    else
        %serialize the array and chop off anything that doesn't fit
        LSBarray = reshape(LSBarray.', 1, []);
        LSBarray = LSBarray(1:floorDiv(length(LSBarray), bits)*bits);
        LSBarray = reshape(LSBarray', bits, []).';
        %convert to a number so we can cast the whole thing to char
        LSBarray = bin2dec(LSBarray)';
        LsbByteArray = LSBarray;
    end
end
```

B: Source Code Files (Livescript Form)

All live scripts and PDFs are included in zip file. The following pages include the source code to generate the figures present in the report with the exception of three that were generated by hand in excel of collated data. Excess pages that printed out thousands of repeated encoded messages were also removed.

Final Project Implementation/Demonstration

LSB Encoding - Basics

LSB encoding is a technique that uses the smallest least significant bit (LSB) of a sequence of samples to convey data, essentially hiding a secondary data stream in the place where interference is least likely to result.

```
clc; close all
%obtain our audio sample of 1 minute
[audio, Fs] = audioread("CantinaBand60.wav", 'native');

%signal to be concealed and how many times to play it
hidden_msg = ['I am a hidden message. You cannot see me, or can you? Repeat: ' newLine
%for demo's sake, fill up the whole file
repetitions = floorDiv(length(audio),length(hidden_msg)*8);

%convert string to a bitstream
tx_bitstream = makestream_string(repmat(hidden_msg, 1, repetitions))
```

```
tx_bitstream =
'010010010010000001100001011011010010000001100001001000000110100001101001011001000110010001100101011011100
```

```
number_of_bits = length(tx_bitstream)
```

```
number_of_bits = 1323000
```

```
number_of_samples = length(audio)
```

```
number_of_samples = 1323000
```

```
data_rate = Fs
```

```
data_rate = 22050
```

Because LSB encoding allows us to add transmit one bit per sample, our data rate is equal to our sample rate. This can be increased by using more than just the LSB. Using the N least significant bits can increase the data rate at the cost of increaaing distortion.

```
%generate a new audio vector using function
output = encode_audiostream_1D(audio, tx_bitstream);
audiowrite('secret_message.wav', output, Fs)
seconds_to_play = 5;
player = audioplayer(audio(1:seconds_to_play*Fs), Fs);
%play(player)

%extract the hidden message
LSBytes = extractLSB(output, 8);
char(LSBytes)
```

```
ans =
```

[illegible]

Increasing Data Rate

Now that we have a method for encoding arbitrary binary streams into the LSB of a signal, we can apply permutations of this technique to do some more interesting things. The first thing we can try to do is increase the data rate. This project uses an example of CD-quality bit depth, or 16-bit audio. We heard very little distortion with only the LSB of the audio being affected, but what about using the 2 least significant bits (LSBs)? 4? Even more? We first need to rewrite our functions to handle this variable.

```

clear;
[audio, Fs] = audioread("CantinaBand60.wav", 'native');

%2x Sample rate, 2 LSBs
hidden_msg = ['How fast can we go? 2xFs bps Repeat: ' newline];
num_LSBs = 2; %number of LSBs to use

%create a bitstream for the whole file
repetitions = floorDiv(length(audio)*num_LSBs,length(hidden_msg)*8);
tx_bitstream = makestream_string(repmat(hidden_msg, 1, repetitions))
length(audio)*num_LSBs
length(tx_bitstream)

%encode
output = encode_audiostream_ND(audio, tx_bitstream, num_LSBs);

%play
seconds_to_play = 5;
player = audioplayer(output(1:seconds_to_play*Fs), Fs);
%play(player)

%decode
LSBytes = extract_N_LSB(output, 8, num_LSBs);
char(LSBytes)
audiowrite('secret_cantina_2LSBs.wav', output, Fs)

```

This sounds fine to my ears. Empirically, it starts to get very noticeable around 10 bits. Let's generate some more wavs

```

clear;
[audio, Fs] = audioread("CantinaBand60.wav", 'native');

%6x Sample rate, 6 LSBs
hidden_msg = ['How fast can we go? 6xFs bps Repeat: ' newline];
num_LSBs = 6; %number of LSBs to use

%create a bitstream for the whole file
repetitions = floorDiv(length(audio)*num_LSBs,length(hidden_msg)*8);
tx_bitstream = makestream_string(repmat(hidden_msg, 1, repetitions))
length(audio)*num_LSBs
length(tx_bitstream)

%encode
output = encode_audiostream_ND(audio, tx_bitstream, num_LSBs);

%play
seconds_to_play = 5;
player = audioplayer(output(1:seconds_to_play*Fs), Fs);
%play(player)

```

```

%decode
LSBytes = extract_N_LSB(output, 8, num_LSBs);
char(LSBytes)
audiowrite('secret_cantina_6LSBs.wav', output, Fs)

```

9 bits

```

clear;
[audio, Fs] = audioread("CantinaBand60.wav", 'native');

%9x Sample rate, 9 LSBs
hidden_msg = ['How fast can we go? 9xFs bps Repeat: ' newline];
num_LSBs = 9; %number of LSBs to use

%create a bitstream for the whole file
repetitions = floorDiv(length(audio)*num_LSBs, length(hidden_msg)*8);
tx_bitstream = makestream_string(repmat(hidden_msg, 1, repetitions))
length(audio)*num_LSBs
length(tx_bitstream)

%encode
output = encode_audiostream_ND(audio, tx_bitstream, num_LSBs);

%play
seconds_to_play = 5;
player = audioplayer(output(1:seconds_to_play*Fs), Fs);
%play(player)

%decode
LSBytes = extract_N_LSB(output, 8, num_LSBs);
char(LSBytes)
audiowrite('secret_cantina_9LSBs.wav', output, Fs)

```

10 bits

```

clear;
[audio, Fs] = audioread("CantinaBand60.wav", 'native');

%10x Sample rate, 10 LSBs
hidden_msg = ['How fast can we go? 10xFs bps Repeat: ' newline];
num_LSBs = 10; %number of LSBs to use

%create a bitstream for the whole file
repetitions = floorDiv(length(audio)*num_LSBs, length(hidden_msg)*8);
tx_bitstream = makestream_string(repmat(hidden_msg, 1, repetitions))
length(audio)*num_LSBs
length(tx_bitstream)

%encode
output = encode_audiostream_ND(audio, tx_bitstream, num_LSBs);

```

```

%play
seconds_to_play = 5;
player = audioplayer(output(1:seconds_to_play*Fs), Fs);
%play(player)

%decode
LSBytes = extract_N_LSB(output, 8, num_LSBs);
char(LSBytes)
audiowrite('secret_cantina_10LSBs.wav', output, Fs)

```

12 bits, quite distorted

```

[audio, Fs] = audioread("CantinaBand60.wav", 'native');

%12x Sample rate, 12 LSBs
hidden_msg = ['How fast can we go? 12x Fs bps Repeat: ' newline];
num_LSBs = 12; %number of LSBs to use

%create a bitstream for the whole file
repetitions = floorDiv(length(audio)*num_LSBs, length(hidden_msg)*8);
tx_bitstream = makestream_string(repmat(hidden_msg, 1, repetitions))
length(audio)*num_LSBs
length(tx_bitstream)

%encode
output = encode_audiostream_ND(audio, tx_bitstream, num_LSBs);

%play
seconds_to_play = 5;
player = audioplayer(output(1:seconds_to_play*Fs), Fs);
%play(player)

%decode
LSBytes = extract_N_LSB(output, 8, num_LSBs);
char(LSBytes)
audiowrite('secret_cantina_12LSBs.wav', output, Fs)

```

Encoding More Interesting Data

In the above section we proved out an implementation of LSB encoding with arbitrary bitstreams and bit-depth, but when we're just sending text all we can do to show off the data rate is repeat the message many times. What if we wanted to send something more interesting? An image is a fun visual piece of information, and also takes far more data than text to represent. Let's use a classic DSP image of peppers for this.

Source for images: Queen Mary University of London

```

clear
%take a look at our peppers

```

```

pep = 512x512 uint8 matrix
    5    65    67    57    66    60    57    55    58    61    66    72    76 ...
   20    84    89    79    88    84    82    81    85    85    85    84    83
   20    88    97    86    93    88    87    87    88    88    88    87    85
   14    86    96    82    86    80    79    78    78    80    83    85    86
   17    90   101    86    90    87    85    83    81    83    87    90    88
   13    87    97    82    90    91    91    87    78    81    85    88    88
    9    81    87    72    83    88    87    80    75    77    80    83    85
   14    84    88    71    82    87    85    76    86    85    84    84    85
   10    78    83    84    78    90    84    79    79    85    85    82    82
   11    79    84    85    78    90    85    80    70    77    82    82    83
    ⋮

```

```
imshow(pep)
```



Peppers is a 512*512 greyscale tiff with 8 bits of depth. Let's calculate the size of an audio file needed to transmit it using a single LSB. Remember: the data rate is dependent on the sample rate of the audio.

```

[audio, Fs] = audioread("CantinaBand60.wav", 'native');
bits_in_image = 512*512 * 8

```

```
bits_in_image = 2097152
```

```
length(audio)
```

```
ans = 1323000
```

It looks like our image contains more bits than our audio has samples. This means we will need to either compress the data, or utilize more LSBs to store it. Because we already have a method of using more LSBs, let's try that. Compression, encryption, and forward error correction are commonly used in steganography but we can save those for another time.

```

%first we need to serialize the image
pep = imread('peppers.tif')
binary_pep = dec2bin(pep);

```

```

binary_pep = reshape(binary_pep.', 1, []); %1s and 0s

%now embed into audio using our past function. We can use 2 for LSB
num_LSBs = 2;
peppered_song = encode_audiostream_ND(audio, binary_pep, num_LSBs);
audiowrite('peppered_cantina.wav', peppered_song, Fs)

%play a sample
seconds_to_play = 5;
player = audioplayer(peppered_song(1:seconds_to_play*Fs), Fs);
%play(player)

%extract serialized image bytes
serialized_pep = extract_N_LSB(peppered_song, 8, num_LSBs)

```

```

serialized_pep = 1x262144
    5    20    20    14    17    13     9    14    10    11    11    10     8 ...

```

```

%we know how many bytes we are looking for
bytes_in_image = 512*512;
serialized_pep = serialized_pep(1:bytes_in_image)

```

```

serialized_pep = 1x262144
    5    20    20    14    17    13     9    14    10    11    11    10     8 ...

```

```

reconstructed_pepper = reshape(uint8(serialized_pep), [512, 512])

```

```

reconstructed_pepper = 512x512 uint8 matrix
    5    65    67    57    66    60    57    55    58    61    66    72    76 ...
   20    84    89    79    88    84    82    81    85    85    85    84    83
   20    88    97    86    93    88    87    87    88    88    88    87    85
   14    86    96    82    86    80    79    78    78    80    83    85    86
   17    90   101    86    90    87    85    83    81    83    87    90    88
   13    87    97    82    90    91    91    87    78    81    85    88    88
     9    81    87    72    83    88    87    80    75    77    80    83    85
   14    84    88    71    82    87    85    76    86    85    84    84    85
   10    78    83    84    78    90    84    79    79    85    85    82    82
   11    79    84    85    78    90    85    80    70    77    82    82    83
   ⋮

```

```

imshow(reconstructed_pepper)

```



```
function [hidden_bits] = makestream_string(message)
    %create a bitstream given a string of arbitrary length
    message = uint8(message);
    message = dec2bin(message);
    %Init array
    hidden_bits = [];

    %Loop
    for i = 1:length(message)
        binaryStr = message(i,:);

        %Make sure we have 8-bits for this.
        if length(binaryStr) == 7
            paddedBinaryStr = ['0', binaryStr];
        else
            paddedBinaryStr = binaryStr;
        end

        %Append each bit to hidden_bits
        for j = 1:8
            bit = paddedBinaryStr(j);
            hidden_bits = [hidden_bits, bit];
        end
    end
end
```

```

function [output] = encode_audiostream_1D(audio, bitstream)
%encode a bitstream into the LSB of an audio file
for i = 1:length(bitstream)
    %set LSB to 0 so we can OR it
    %00...001 is -2 for int16
    audio(i) = bitand(audio(i), int16(-2));
    %set LSB using OR and the bitStream

    audio(i) = bitor(audio(i), typecast(uint16(bin2dec(bitstream(i))), 'int16'));
end
output = audio;
end

function [output] = encode_audiostream_ND(audio, bitstream, n_bits)
%encode N-dimensionally
%encode a bitstream into the N LSBs of an audio file
zero_out = repmat('0', 1, n_bits);
one_out = repmat('1', 1, 16-n_bits);
binstr = [one_out zero_out];
ander = typecast(uint16(bin2dec(binstr)), 'int16');
audio=audio(1:floorDiv(length(bitstream),n_bits));

for i = 1:length(audio)
    bit_idx_start = (i-1) * n_bits + 1; %3
    bit_idx_end = i*n_bits; %4
    %set LSBs to 0 so we can OR it
    %00...001 is -2 for int16
    audio(i) = bitand(audio(i), int16(ander));
    %set LSB using OR and the bitStream
    bitsection = bitstream(bit_idx_start:bit_idx_end);
    audio(i) = bitor(audio(i), typecast(uint16(bin2dec(bitsection)), 'int16'));
end
output = audio;
end

function [LsbByteArray] = extractLSB(audio,bits)
%take the LSB from audio and chunk it into bytes the size of
%the bits argument.
LSBarray = bitand(audio(1:end), int16(1));
%only return the a number of bits that are divisibile by data size
LSBarray = LSBarray(1:floorDiv(length(LSBarray), bits)*bits);
LSBarray = reshape(LSBarray, bits, []);
%a lot going on in this one-liner, but boils down to
%transposing the array and converting the 0s and 1s to 8 bit ints
LsbByteArray = bin2dec(num2str(LSBarray'))';
end

function [LsbByteArray] = extract_N_LSB(audio,bits,num_LSBs)

```

```

%take the LSB from audio and chunk it into bytes the size of
%the bits argument. Do this for N LSBs
zero_out = repmat('0', 1, 16-num_LSBs);
one_out = repmat('1', 1, num_LSBs);
binstr = [zero_out one_out];

LSBarray = dec2bin(bitand(audio(1:end), typecast(uint16(bin2dec(binstr)), 'int16'))
%serialize the array and chop off anything that doesn't fit
LSBarray = reshape(LSBarray.', 1, []);
LSBarray = LSBarray(1:floorDiv(length(LSBarray), bits)*bits);
%chunk it into 8-bit parts
LSBarray = reshape(LSBarray', bits, []).';
%convert to a number so we can cast the whole thing to char
LSBarray = bin2dec(LSBarray)';
LsbByteArray = LSBarray;
end

```



```
function [LsbByteArray] = extract_N_LSB(audio,bits,num_LSBs)
    %take the LSB from audio and chunk it into bytes the size of
    %the bits argument. Do this for N LSBs
    zero_out = repmat('0', 1, 16-num_LSBs);
    one_out = repmat('1', 1, num_LSBs);
    binstr = [zero_out one_out];

    LSBarray = dec2bin(bitand(audio(1:end), typecast(uint16(bin2dec(binstr)), 'int
    %serialize the array and chop off anything that doesn't fit
    LSBarray = reshape(LSBarray.', 1, []);
    LSBarray = LSBarray(1:floorDiv(length(LSBarray), bits)*bits);
    %chunk it into 8-bit parts
    LSBarray = reshape(LSBarray', bits, []).';
    %convert to a number so we can cast the whole thing to char
    LSBarray = bin2dec(LSBarray)';
    LsbByteArray = LSBarray;
end
```

```

clear
[audio, Fs] = audioread("peppered_cantina.wav", 'native');
player = audioplayer(audio, Fs);
%play(player)

%appropriate audio
serialized_pep = extract_N_LSB(audio, 8, 2);
%use bits_in_image calculated from earlier to slice what is needed
bytes_in_image = 512*512;
serialized_pep = serialized_pep(1:bytes_in_image)

```

```

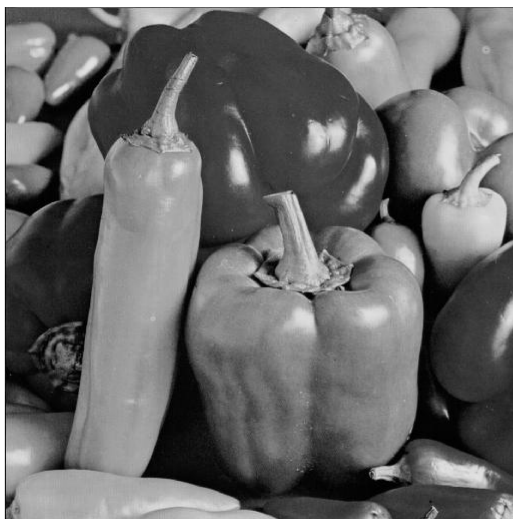
serialized_pep = 1x262144
               5    20    20    14    17    13    9    14    10    11    11    10    8 ...

```

```

reconstructed_pepper = reshape(uint8(serialized_pep), [512, 512]);
imshow(reconstructed_pepper)

```



```

%filtered audio
%this comes from assignment 8, and I've reused much of my code with a
%different sample rate
passband_edge = 700; %Hz
stopband_edge = 1000; %Hz
Fc = 0.5 * (passband_edge + stopband_edge); % Hz
wc_sansPi = 2*(Fc/Fs);
wc = wc_sansPi * pi;
transi_BW = 2*pi* ((stopband_edge-passband_edge)/Fs);
M_min = (12*pi)/transi_BW; %441, good odd length filter

>window it
wc_win = wc;
M = -220:220;
window = blackman(441);

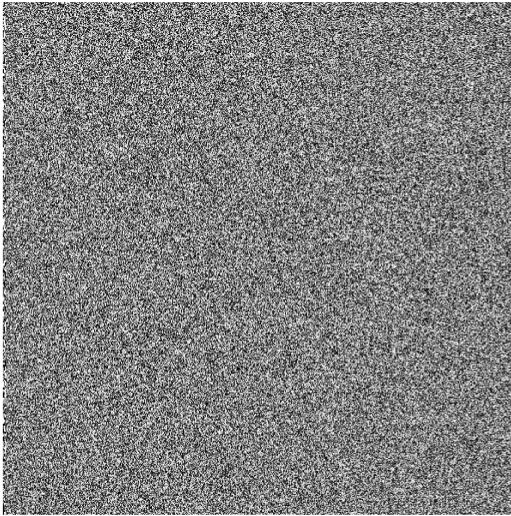
```

```

h = sin(wc_win*M)./(pi*M);
h(221) = wc_win/pi;
h_windowed = h .* window';

%filter
filtered_pep = conv(audio,h_windowed, 'same');
serialized_filtered_pep = extract_N_LSB(int16(filtered_pep), 8, 2);
reconstructed_filtered_pep = reshape(uint8(serialized_filtered_pep), [512, 512]);
imshow(reconstructed_filtered_pep)

```



```

function [LsbByteArray] = extract_N_LSB(audio,bits,num_LSBs)
    %take the LSB from audio and chunk it into bytes the size of
    %the bits argument. Do this for N LSBs
    zero_out = repmat('0', 1, 16-num_LSBs);
    one_out = repmat('1', 1, num_LSBs);
    binstr = [zero_out one_out];

    LSBarray = dec2bin(bitand(audio(1:end), typecast(uint16(bin2dec(binstr)), 'int16')), 'int16')
    %the special case of 8 does not return a full 8 bit binary string, it
    %drops the leading 0. We can send it directly to output
    if num_LSBs == 8
        LsbByteArray = bin2dec(LSBarray)';
    else
        %serialize the array and chop off anything that doesn't fit
        LSBarray = reshape(LSBarray.', 1, []);
        LSBarray = LSBarray(1:floorDiv(length(LSBarray), bits)*bits);
        LSBarray = reshape(LSBarray', bits, []).';
        %convert to a number so we can cast the whole thing to char
        LSBarray = bin2dec(LSBarray)';
        LsbByteArray = LSBarray;
    end
end

```

```
[audio, Fs] = audioread("CantinaBand60.wav", 'native');  
t = linspace(0, length(audio)/Fs, length(audio));  
plot(t(1:Fs), audio(1:Fs)); title("Original Audio – First Second");  
xlabel('time(seconds)'); ylabel('Magnitude (16-bit int)')
```

Demo 1

Read in the 1-LSB wav file and extract the LSBs from it, reconstructing it into a character array for display.

```
[original_audio, Fs] = audioread("CantinaBand60.wav", 'native');  
[audio, Fs] = audioread("secret_message.wav", 'native');  
seconds_to_play = 5
```

```
seconds_to_play = 5
```

```
player = audioplayer(audio(1:seconds_to_play*Fs), Fs);  
play(player)  
LSBytes = extract_N_LSB(audio, 8, 1);  
length(LSBytes)
```

```
ans = 165375
```

```
length(audio)
```

```
ans = 1323000
```

```
char(LSBytes)
```

```
ans =  
'I am a hidden message. You cannot see me, or can you? Repeat:  
I am a hidden message. You cannot see me, or can you? Repeat:  
I am a hidden message. You cannot see me, or can you? Repeat:  
I am a hidden message. You cannot see me, or can you? Repeat:  
I am a hidden message. You cannot see me, or can you? Repeat:'
```



```

for num_LSBs = num_LSBs_loop
    [original_audio, Fs] = audioread("CantinaBand60.wav", 'native');
    filename = "secret_cantina_" + num2str(num_LSBs) + "LSBs.wav"
    [audio, Fs] = audioread(filename, 'native');
    LSBytes = extract_N_LSB(audio, 8, num_LSBs);
    number_of_LSBs_used = num_LSBs
    message_size = length(LSBytes)
    data_rate = length(LSBytes)*8/ (length(audio)/Fs)
    %length(audio)
    %char(LSBytes)
    original_audio = original_audio(1:length(audio));
    noise = double(original_audio - audio);
    SNR = snr(double(original_audio), noise)
end

```

```

filename =
"secret_cantina_1LSBs.wav"
number_of_LSBs_used = 1
message_size = 165375
data_rate = 22050
SNR = 66.3876
filename =
"secret_cantina_2LSBs.wav"
number_of_LSBs_used = 2
message_size = 330714
data_rate = 44100
SNR = 59.3887
filename =
"secret_cantina_3LSBs.wav"
number_of_LSBs_used = 3
message_size = 496089
data_rate = 6.6150e+04
SNR = 53.0238
filename =
"secret_cantina_4LSBs.wav"
number_of_LSBs_used = 4
message_size = 661466
data_rate = 88200
SNR = 47.1970
filename =
"secret_cantina_5LSBs.wav"
number_of_LSBs_used = 5
message_size = 826841
data_rate = 1.1025e+05
SNR = 40.8933
filename =
"secret_cantina_6LSBs.wav"
number_of_LSBs_used = 6
message_size = 992217
data_rate = 1.3230e+05
SNR = 35.1120
filename =
"secret_cantina_7LSBs.wav"
number_of_LSBs_used = 7
message_size = 1157593
data_rate = 1.5435e+05
SNR = 28.8095
filename =
"secret_cantina_8LSBs.wav"
number_of_LSBs_used = 8

```

```

message_size = 1322970
data_rate = 176400
SNR = 24.0021
filename =
"secret_cantina_9LSBs.wav"
number_of_LSBs_used = 9
message_size = 1488345
data_rate = 1.9845e+05
SNR = 16.7563
filename =
"secret_cantina_10LSBs.wav"
number_of_LSBs_used = 10
message_size = 1653716
data_rate = 2.2050e+05
SNR = 10.9098
filename =
"secret_cantina_11LSBs.wav"
number_of_LSBs_used = 11
message_size = 1819115
data_rate = 2.4255e+05
SNR = 4.5564
filename =
"secret_cantina_12LSBs.wav"
number_of_LSBs_used = 12
message_size = 1984476
data_rate = 264600
SNR = -1.8585
filename =
"secret_cantina_13LSBs.wav"
number_of_LSBs_used = 13
message_size = 2149875
data_rate = 286650
SNR = -8.7454

```

Demo 3

Quality of pepper image

```
num_LSBs = 2
```

```
num_LSBs = 2
```

```

[original_audio, Fs] = audioread("CantinaBand60.wav", 'native');
[audio, Fs] = audioread('peppered_cantina.wav', 'native');
LSBytes = extract_N_LSB(audio, 8, num_LSBs);
number_of_LSBs_used = num_LSBs

```

```
number_of_LSBs_used = 2
```

```
message_size = length(LSBytes)
```

```
message_size = 262144
```

```
data_rate = length(LSBytes)*8/ (length(audio)/Fs)
```

```
data_rate = 44100
```

```
%length(audio)
```

```

%char(LSBytes)
original_audio = original_audio(1:length(audio));
noise = double(original_audio - audio);
SNR = snr(double(original_audio), noise)

```

SNR = 58.8070

```

function [LsbByteArray] = extract_N_LSB(audio,bits,num_LSBs)
    %take the LSB from audio and chunk it into bytes the size of
    %the bits argument. Do this for N LSBs
    zero_out = repmat('0', 1, 16-num_LSBs);
    one_out = repmat('1', 1, num_LSBs);
    binstr = [zero_out one_out];

    LSBarray = dec2bin(bitand(audio(1:end), typecast(uint16(bin2dec(binstr)), 'int16'))
    %the special case of 8 does not return a full 8 bit binary string, it
    %drops the leading 0. We can send it directly to output
    if num_LSBs == 8
        LsbByteArray = bin2dec(LSBarray)';
    else
        %serialize the array and chop off anything that doesn't fit
        LSBarray = reshape(LSBarray.', 1, []);
        LSBarray = LSBarray(1:floorDiv(length(LSBarray), bits)*bits);
        LSBarray = reshape(LSBarray', bits, []).';
        %convert to a number so we can cast the whole thing to char
        LSBarray = bin2dec(LSBarray)';
        LsbByteArray = LSBarray;
    end
end

```